
Software Design Specification
for
SERS – Student Elective Registration System
Version 1.0

Document Information

Title:	SERS – Student Elective Registration System SDS		
Project Manager:	Murali Krishna (Instructor)	Document Version No:	1.0
Prepared By:	—	Document Version Date:	April 2026
Preparation Date:	April 2026		

Version History

Ver.	Date	Revised By	Description	Filename
1.0	Apr 2026	(All Members)	Initial SDS: architecture, design model, data view, QoS, exception handling.	SERS_SDS_v1.0.docx

1 Introduction

This section provides a comprehensive overview of the Software Design Specification (SDS) for our software, the Student Elective Registration System (SERS). It establishes the purpose of the document, defines its scope relative to the SERS project, provides a consolidated glossary of all terms and acronyms used, and lists all reference materials consulted during design. Readers are expected to familiarise themselves with this section before proceeding to the detailed design views in subsequent sections.

1.1 Purpose

This Software Design Specification describes the complete software design for SERS, which is a web-based elective course registration platform built for third and fourth-year engineering students at Mahindra University. The SDS translates the functional and non-functional requirements documented in the SERS Software Requirements Specification (SRS, Version 1.0, March 2026) into an implementable design. Specifically, this document:

- Defines the three-tier architectural decomposition of SERS into a React.js frontend, a Spring Boot backend, and a PostgreSQL data layer hosted on Supabase.
- Describes all significant design classes, modules, and their relationships through UML class diagrams and sequence diagrams.
- Documents the data model, including all seven database tables, their attributes, data types, primary/foreign keys, and integrity constraints.
- Specifies the system's exception-handling strategy, configurable deployment parameters, and quality-of-service provisions (availability, security, load handling, and monitoring).
- Serves as a binding reference for all developers, testers, and reviewers involved in the construction and evaluation of SERS.

The primary audiences for this document are: (1) the development team (), who use the design model as the blueprint for implementation; (2) the course instructor, who reviews the design for completeness and architectural soundness; and (3) evaluators or peer-reviewers assessing the system against its stated requirements. Developers should read Sections 3 and 4 for implementation guidance; evaluators should focus on Sections 2, 5, and 8.

1.2 Scope

This SDS applies to Version 1.0 of the SERS software as developed for the Software Engineering course at Mahindra University. SERS is a self-contained platform that automates elective course selection and allocation for students enrolled in third and fourth year across approximately four to five engineering branches. The system is not a generic Learning Management System, it is purpose-built for the elective registration lifecycle.

The design described in this document covers the following functional boundaries:

- Student-facing features: Secure authentication via university credentials, personalised virtual timetable dashboard, real-time elective browsing, draft (Preview) and confirmation of course selections, and viewing final allocation results.

- Administrator-facing features: Elective course CRUD operations, seat capacity configuration, allocation process execution, manual override (force-allocation), and aggregate analytics dashboards.
- Professor-facing features: View of assigned courses and enrolled student rosters.
- Core business logic: the Conflict Checker (schedule clash detection), Seat Limiter (capacity enforcement), Category Limiter (elective-type quotas), and the Priority Score allocation algorithm (CGPA + year + timestamp weighting).
- System infrastructure: Spring Boot layered architecture (Controller / Service / Repository / Entity / DTO), PostgreSQL schema with seven relational tables, RESTful API communication, JWT-based session management, RBAC security, and Supabase cloud deployment.

This document does not cover: General university identity provider configuration beyond the authentication interface, physical network infrastructure, mobile native applications (the system is browser-accessible on mobile but not a native app), or future extensions such as multi-language support or additional academic branches beyond the initial scope.

1.3 Definitions, Acronyms, and Abbreviations

The following table defines all terms, acronyms, and abbreviations used in this SDS. For terms defined in the SRS, the definitions are reproduced here for self-contained reference.

Term / Acronym	Definition
CGPA	Cumulative Grade Point Average — a student's weighted academic score on a 0.00–10.00 scale, used as one component of the Priority Score.
COMET	Concurrent Object Modeling and Architectural Design Method — the development methodology adopted for SERS.
Confirmed Status	A permanent elective selection state that decrements the course's available seat count in the database.
Conflict Checker	The backend service logic that prevents a student from confirming an elective whose scheduled days and times overlap with any existing mandatory or confirmed course.
CORS	Cross-Origin Resource Sharing — an HTTP mechanism that allows the React frontend to communicate with the Spring Boot backend across different origins.
DTO	Data Transfer Object — a plain Java object used to carry data between layers without exposing JPA entities.
IPv4 Forced Resolution	A JVM-level configuration flag (-Djava.net.preferIPv4Stack=true) used to ensure the application connects to Supabase on university networks that block IPv6.
JPA	Java Persistence API — the specification used within Spring Boot for database interaction.

N+1 Problem	A database anti-pattern where a loop triggers one query per record; prevented in SERS via JOIN-fetched JPQL queries.
ORM	Object-Relational Mapping — the technique (via Hibernate/JPA) that maps Java entity classes to PostgreSQL tables.
PostgreSQL	An open-source, ACID-compliant relational database management system used as the data layer.
Preview Status	A transient elective selection state that tests a course visually on the student's timetable without consuming a database seat.
Priority Score	A composite numeric value calculated during allocation from a student's CGPA, year of study, and preference submission timestamp, used to rank students for seat assignment.
RBAC	Role-Based Access Control — a security model restricting system features based on a user's role (Student, Admin, Professor).
REST API	Representational State Transfer Application Programming Interface; the communication protocol between frontend and backend, using JSON payloads over HTTPS.
SDS	Software Design Specification — this document.
SERS	Student Elective Registration System — the web-based platform described in this document.
SPA	Single Page Application — the React frontend architecture in which one HTML page is served and views are updated dynamically without full-page reloads.
Spring Boot	A Java framework for rapidly building production-grade RESTful backend services.
SRS	Software Requirements Specification — the antecedent requirements document for SERS.
Supabase	A cloud-hosted PostgreSQL provider used for database hosting and connection pooling.
XSS	Cross-Site Scripting — a web vulnerability where malicious scripts are injected into pages; mitigated by input sanitisation in SERS.

1.4 References

The following documents and resources were consulted during the preparation of this SDS:

- [1] (). SERS Software Requirements Specification, Version 1.0. Mahindra University, March 2026. (Primary antecedent document for this SDS.)
- [2] IEEE. IEEE 29148-2018 – Systems and Software Engineering: Life Cycle Processes – Requirements Engineering. IEEE, 2018.

- [3] Gomaa, H. Designing Concurrent, Distributed, and Real-Time Applications with UML. Addison-Wesley, 2000. (Primary reference for the COMET methodology adopted by the project.)
- [4] Object Management Group (OMG). Unified Modeling Language (UML) Specification, Version 2.5.1. OMG, 2017. <https://www.omg.org/spec/UML/>
- [5] Spring Framework Team. Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/> (Accessed April 2026.)
- [6] React Team. React Documentation. <https://react.dev> (Accessed April 2026.)
- [7] The PostgreSQL Global Development Group. PostgreSQL 16 Documentation. <https://www.postgresql.org/docs/16/> (Accessed April 2026.)
- [8] Supabase Inc. Supabase Documentation — Database & Connection Pooling. <https://supabase.com/docs> (Accessed April 2026.)
- [9] Hibernate ORM Team. Hibernate ORM 6 User Guide. <https://hibernate.org/orm/documentation/> (Accessed April 2026.)
- [10] OWASP Foundation. OWASP Top Ten Web Application Security Risks. <https://owasp.org/www-project-top-ten/> (Accessed April 2026.)

2 Use Case View

This section identifies and describes all significant use cases of the Student Elective Registration System. A use case is significant as it represents core system functionality and exercises multiple design components to show system's business logic. SERS supports three actors: Student, Administrator, and Professor, interacting with ten use cases across five functional subsystems: Authentication, Student Elective Management, Business Logic Enforcement, Administrative Control, and Faculty Oversight. Use case diagrams are provided at the overall system level and at a detailed level for the two most design-critical use cases (UC4 and UC7).

2.1 Use Case

Figure 2.1 presents the overall use case diagram for SERS, illustrating all actors (Student, Admin, Professor) and their relationships with the ten use cases. Included use cases (business rule checks invoked automatically) are shown with dashed «include» arrows; extended use cases (conditional error flows) use «extend» arrows.

The most design-critical student-facing use case is shown in Figure 2.2 below. It coordinates the Conflict Checker, Seat Limiter, and Category Limiter business rules, and manages the two-phase selection lifecycle (Preview → Confirmed).

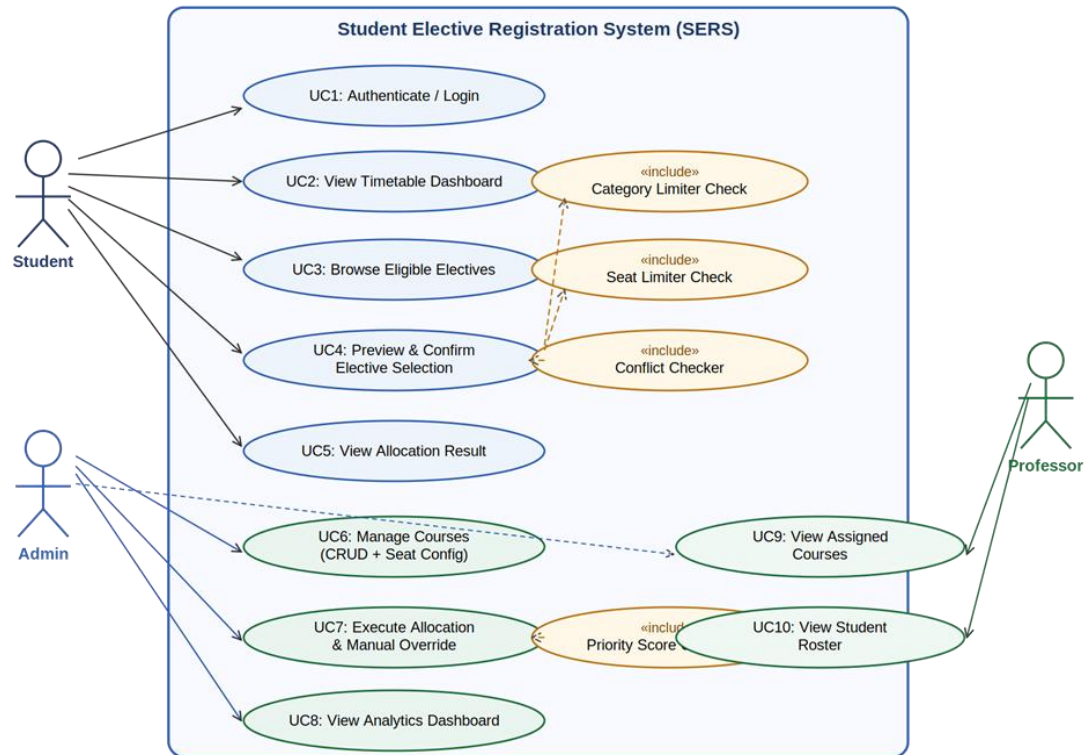


Figure 2.1 – SERS Overall Use Case Diagram

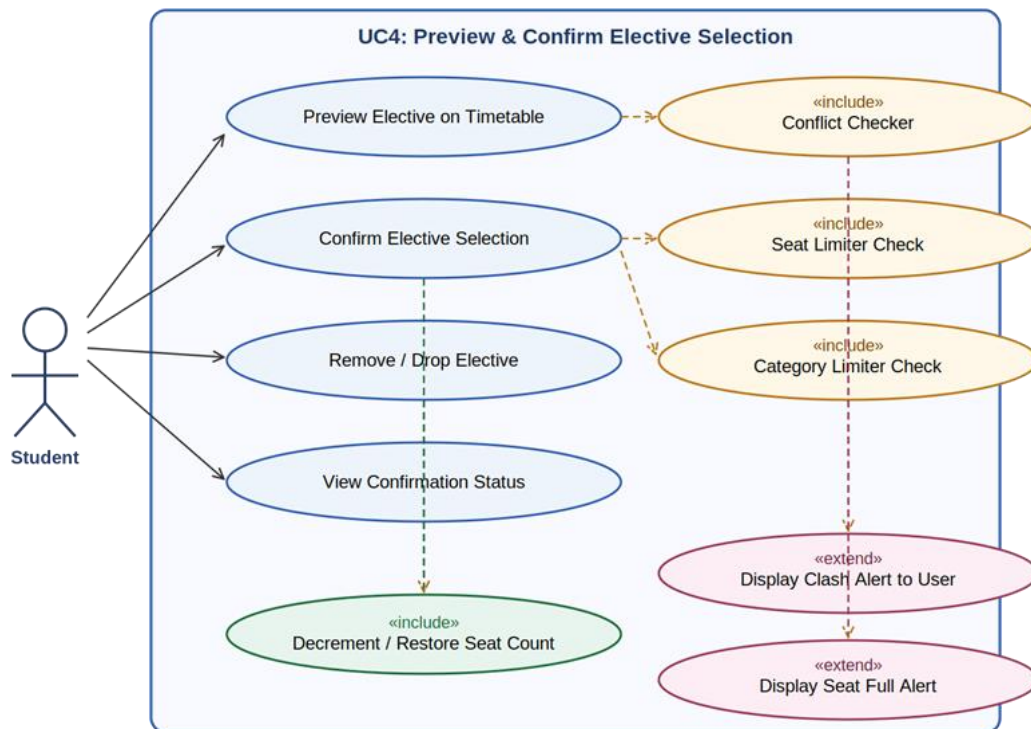


Figure 2.2 – UC4: Preview & Confirm Elective Selection

UC1 – Authenticate / Login: This use case handles secure user authentication for Students, Administrators, and Professors. Users submit institutional credentials via the frontend, which are validated by the backend. Upon successful verification, a JWT token is issued and the user is redirected to their role-specific dashboard. The system ensures secure session handling, prevents user enumeration, and gracefully manages errors like invalid credentials or server unavailability.

UC2 – View Timetable Dashboard: This allows an authenticated student to view a personalized weekly timetable displaying mandatory courses along with previewed and confirmed electives. The backend aggregates course data, and the frontend renders it in a grid with visual distinctions. Real-time seat availability is maintained through periodic polling, ensuring up-to-date information.

UC3 – Browse Eligible Electives: Students can browse a filtered list of electives based on eligibility criteria such as branch, year, seat availability, and constraints. The system dynamically excludes ineligible or already selected courses and displays detailed course information. It also handles cases like full courses or category limits with appropriate UI indicators.

UC4 – Preview & Confirm Elective Selection: This critical use case manages a two-phase elective selection process: Preview and Confirm. During preview, no seat is consumed, while confirmation triggers validations (conflict, seat, category limits) and updates the database atomically. It also supports removal of selections and handles concurrency issues like simultaneous seat allocation attempts.

UC5 – View Allocation Result: Students can view their final elective allocation after the admin runs the allocation process. The system retrieves allocation data and displays either assigned courses or status messages (e.g., waitlisted/unallocated). It also informs users if allocation results are not yet available.

UC6 – Manage Courses (CRUD + Seat Configuration): Administrators can create, update, view, and deactivate courses while configuring seat capacities. The system enforces validations like unique course codes, valid faculty assignments, and seat constraints. Changes are reflected immediately and affect student-facing functionalities.

UC7 – Execute Allocation & Manual Override: This key administrative use case executes a batch allocation algorithm based on a computed priority score (using CGPA, year, etc.). Students are allocated seats in ranked preference order. Admins can also manually override allocations, even bypassing constraints, with all actions logged for audit purposes.

UC8 – View Analytics Dashboard: Administrators can access a real-time analytics dashboard showing system metrics like total registrations, selection counts, and course fill rates. Data is aggregated from multiple tables and visualized using charts and summary statistics.

UC9 – View Assigned Courses: Professors can view all courses assigned to them based on their faculty ID. The system retrieves active courses along with enrollment data and presents them in a structured format for easy access.

UC10 – View Student Roster: Professors can view detailed student rosters for their assigned courses. The system ensures proper authorization and displays student information such as name, roll number, branch, year, and CGPA in a structured, sortable format.

3 Design Overview

3.1 Design Goals and Constraints

Goals: The primary design goal of the Student Elective Registration System (SERS) is to provide a highly responsive, conflict-free registration experience under heavy concurrent user load. The architecture is designed to strictly enforce academic business rules, specifically:

- **Zero-Conflict Scheduling:** The system must algorithmically prevent students from confirming electives that overlap with their fixed core classes or other confirmed electives.
- **Strict Resource Allocation:** The system must maintain strict ACID properties to ensure that course capacities (`max_seats`) and individual student category limits (`max_core_electives`, `max_hum_electives`) are never exceeded, even during peak enrollment windows.
- **Separation of Concerns:** The application must maintain a strict boundary between the presentation layer and the business logic layer, allowing independent scaling and maintenance.

Design and Implementation Constraints: The design and implementation of SERS are heavily influenced by the deployment environment and specific technical limitations:

- **Network and Infrastructure Constraints:**
 - **Port Blocking:** Due to strict university firewall configurations blocking the standard PostgreSQL port (5432), the database connection architecture is constrained to route through a Supabase transaction pooler on port 6543.
 - **DNS/IPv6 Resolution:** The university network's DNS forces IPv6 routing, which causes connection timeouts for the Spring Boot backend. Consequently, the system execution is constrained to explicitly force IPv4 routing via the JVM argument `-Djava.net.preferIPv4Stack=true`.
 - **Connection Pooling Conflicts:** The use of a cloud-based transaction pooler (PgBouncer via Supabase) imposes a strict constraint on the Hibernate ORM layer. Server-side prepared statements must be disabled (via `prepareThreshold=0`) to prevent statement caching collisions across pooled connections.
- **Data and Formatting Constraints:**
 - **Temporal Data Standardization:** To ensure the React frontend's Timetable component accurately renders afternoon and evening classes without silent failures, all temporal data in the persistence layer is strictly constrained to a 24-hour standard (HH:MM format).
- **Development Tools and Strategy:**
 - **Strategy:** The system utilizes a Single Page Application (SPA) strategy using RESTful APIs to communicate between the client and server.
 - **Backend:** Java 17, Spring Boot 3.2.x, Spring Data JPA (Hibernate), and Maven.
 - **Frontend:** React.js, Axios (for HTTP requests), and Recharts (for administrative data visualization).
 - **Database:** PostgreSQL.

3.2 Design Assumptions

The architecture and design of the Student Elective Registration System (SERS) are based on the following key assumptions, which significantly impact the implementation strategy:

- **Cloud Infrastructure and Connection Pooling:** It is assumed that the external database provider (Supabase/PostgreSQL) will maintain high availability and successfully manage concurrent transactional load via its connection pooler (PgBouncer). The backend design assumes that server-side prepared statements can remain disabled (`prepareThreshold=0`) without severe performance degradation.
- **Network Environment:** The design assumes that the system will frequently be accessed from within the university's restricted network. Consequently, it is assumed that the JVM-level IPv4 enforcement (`-Djava.net.preferIPv4Stack=true`) will remain a necessary and effective workaround for the university's IPv6 DNS resolution issues.
- **Data Integrity and Temporal Formatting:** A core assumption of the conflict-checking algorithm (`hasConflict()`) and the `React Timetable.jsx` rendering logic is that all temporal data (course `start_time` and `end_time`) injected into the database strictly adheres to a 24-hour standard (HH:MM). The design assumes no legacy 12-hour format data will be present, as it would cause silent visual failures on the frontend.
- **Client-Side Processing Capabilities:** The system assumes that users are accessing the application via modern web browsers with JavaScript enabled. Because the frontend relies on short-polling (`setInterval` via `Axios`) to fetch real-time seat availability, it assumes the client device can handle lightweight, continuous asynchronous background requests without aggressive browser throttling.
- **Authentication Boundary (MVP Scope):** For this version of the software, it is assumed that the basic role-based matching implemented in the `AuthController` (Student, Admin, Professor) is sufficient for establishing session identity. It is assumed that stateless token-based authentication (e.g., JWT) or OAuth integration is out of scope for the current design phase and will be handled by the deployment environment's broader security perimeter.

3.3 Significant Design Packages

The SERS application utilizes a strict 3-tier architecture consisting of the Presentation, Application/Business, and Data layers. The design model is decomposed into the following significant hierarchical packages:

Backend Design Packages (Spring Boot: `com.sers.sers_backend`) The backend is organized into layers where dependencies flow downward, preventing circular dependencies and ensuring the business logic remains isolated from the database and UI concerns.

- **Controller (API Layer):** This package contains the RESTful endpoints (`StudentController`, `CourseController`, `AdminController`, `ProfessorController`, `AuthController`) that handle incoming HTTP requests from the React frontend.
 - **Dependencies:** It depends on the service package for business logic and the `dto` package for data exchange.

- **Service (Business Logic Layer):** The core "brain" of the application (StudentService, CourseService, AdminService, ProfessorService, AuthService). This layer implements complex rules such as the hasConflict() schedule checking and seat limit enforcement.
 - **Dependencies:** It depends on the repository package for data access and uses entity and dto classes for data mapping.
- **Repository (Data Access Layer):** Interfaces that extend Spring Data JPA (StudentRepository, CourseRepository, etc.) to provide an abstraction over PostgreSQL queries.
 - **Dependencies:** It depends strictly on the entity package.
- **Entity (Domain Model Layer):** Contains the persistent JPA models (Student, Course, ElectiveSelection, Allocation) that map directly to the PostgreSQL database tables.
- **Dto (Data Transfer Objects):** Lightweight objects (CourseDTO, LoginRequest, SelectionRequest) used to transfer data between the frontend and backend without exposing sensitive entity fields like passwords.
- **Config (Infrastructure Layer):** Contains system-level settings, including SecurityConfig.java for endpoint protection and CorsConfig.java to allow communication from localhost:3000.
- **Frontend Design Packages (React SPA: src/)** The frontend is structured to promote component reusability and maintainable state management:
- **Pages (View Containers):** High-level components (StudentDashboard, AdminDashboard, ProfessorDashboard, Login) that manage page-specific state and route logic.
- **Components (Reusable UI Elements):** Small, focused components like Timetable, ElectiveCard, CourseModal, and Toast that handle specific visual tasks.
- **API (Network Layer):** Contains api.js, which centralizes all Axios-based service calls. This ensures that UI components do not have hardcoded URLs and makes it easier to update the BASE API endpoint.

Overall Dependency Hierarchy The system follows a linear execution flow:

1. **User Action** in pages/components.
2. **API Call** via api.js.
3. **Request Handling** in Spring Boot controller.
4. **Business Rule Validation** in service.
5. **Persistence** via repository to the **Supabase PostgreSQL** database.

3.4 Dependent External Interfaces

The table below lists the public interfaces this design requires from other modules or applications.

External Application	Module Using the Interface	Functionality/Description
Supabase Cloud (PostgreSQL Engine)	SERS-backend (specifically the Repository layer)	This is the primary persistent data store for the application. The backend depends on this interface to execute all CRUD operations and maintain transactional integrity for course seat allocations.

University Network Infrastructure (DNS & Firewall)	JVM/ Spring Boot Network Module	The application depends on the organizational environment's network configuration. Specifically, it requires an open path through port 6543 and stable IPv4 routing to reach the cloud database.
Client Web Browser (User Agent)	SERS-frontend	The React application depends on the external browser environment (e.g., Chrome, Opera) to interpret JavaScript and execute API calls via the Axios library to communicate with the backend server.

3.5 Implemented Application External Interfaces (and SOA web services)

The table below lists the implementation of public interfaces this design makes available for other applications.

Interface Name	Module Implementing the Interface	Functionality/Description
User Authentication API (/api/auth)	AuthController	Implements the primary security gateway for the system. It validates user credentials and returns role-specific metadata (Student, Admin, or Professor) to establish session context.
Elective Selection API (/api/student/selection)	StudentController	Provides the interface for transactional enrolment. It processes user requests to "Preview" or "Confirm" electives, directly executing the business logic for seat limits and schedule conflicts.
Course Catalog API (/api/courses)	CourseController	A RESTful interface that allows administrative modules to manage the master list of courses. It supports standard operations to create, update, or deactivate curriculum data.
Administrative Statistics API (/api/admin/dashboard)	AdminController	Publishes aggregated system data, such as total enrolments and course-specific fill rates. This interface provides the raw data used by monitoring dashboards for organizational oversight.

4 Logical View

The Logical View of the SERS application provides a detailed look at the system's internal structure and the collaborative behavior of its components. The design is organized into a **3-tier layered architecture**, ensuring a strict separation of concerns and enhancing maintainability.

The Design Layers:

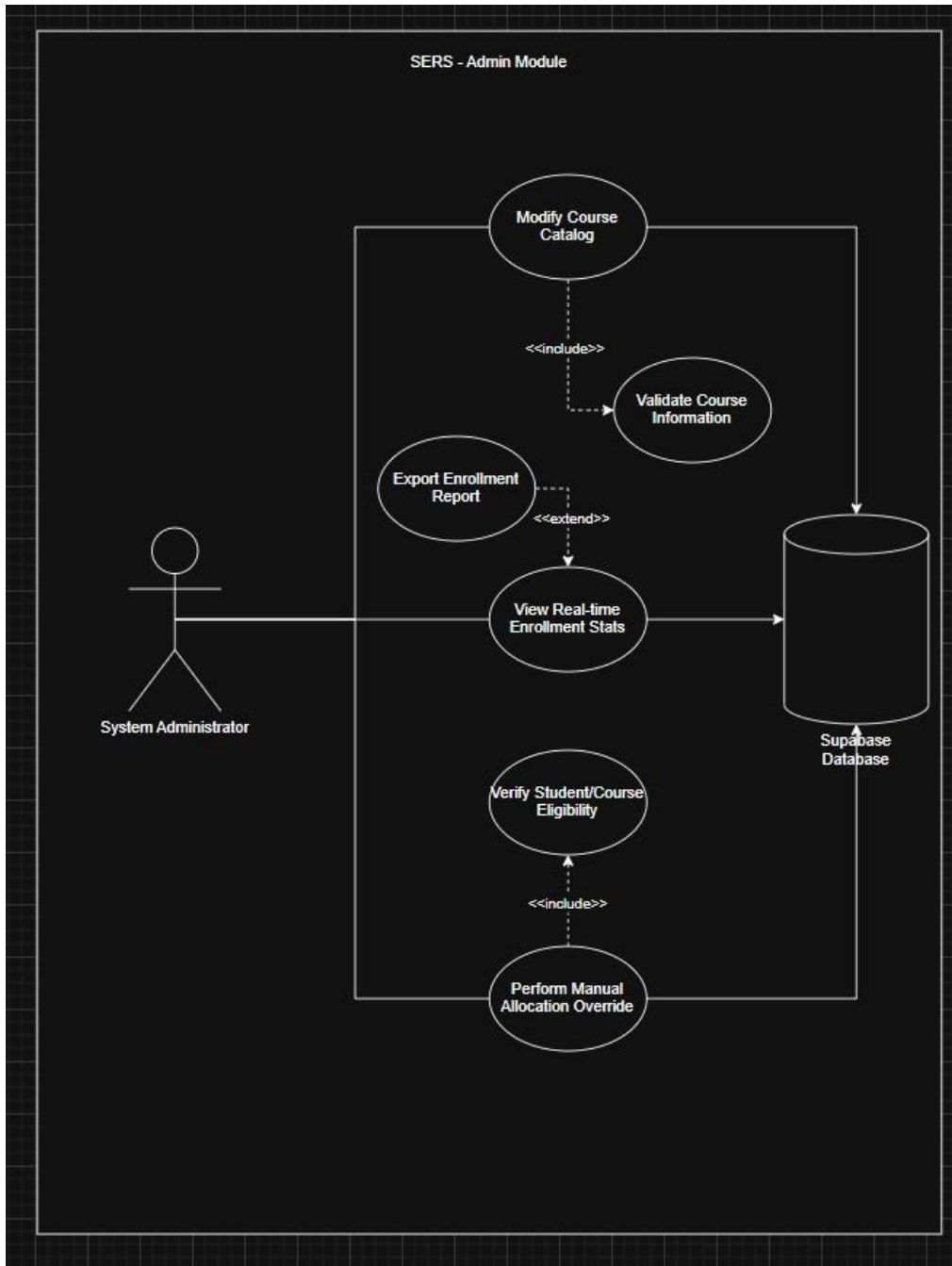
Presentation Layer (Frontend): Consists of React-based SPA (Single Page Application) modules. This layer is responsible for capturing administrative inputs and student selections, managing local state, and providing visual feedback via dashboards.

Application/Business Logic Layer (Backend): Implemented using the Spring Boot framework. This layer orchestrates the system's core logic, such as the `hasConflict()` schedule checking and the Priority Score calculation. It acts as the intermediary, translating user actions into database transactions.

Data Persistence Layer (Database): Utilizes the Supabase PostgreSQL engine. This layer manages the structural integrity of the data, handling the physical storage of student records, course catalogs, and allocation results.

Interaction Philosophy: Control flows from the top (Presentation) down through the Business layer to the Data layer. Each layer only interacts with the one directly below it through well-defined interfaces (REST APIs and Repository interfaces). This layered approach ensures that the "key use cases" identified in Section 2—such as manual overrides and elective confirmations—are executed in a predictable, transactional, and secure manner.

4.1 Design Model

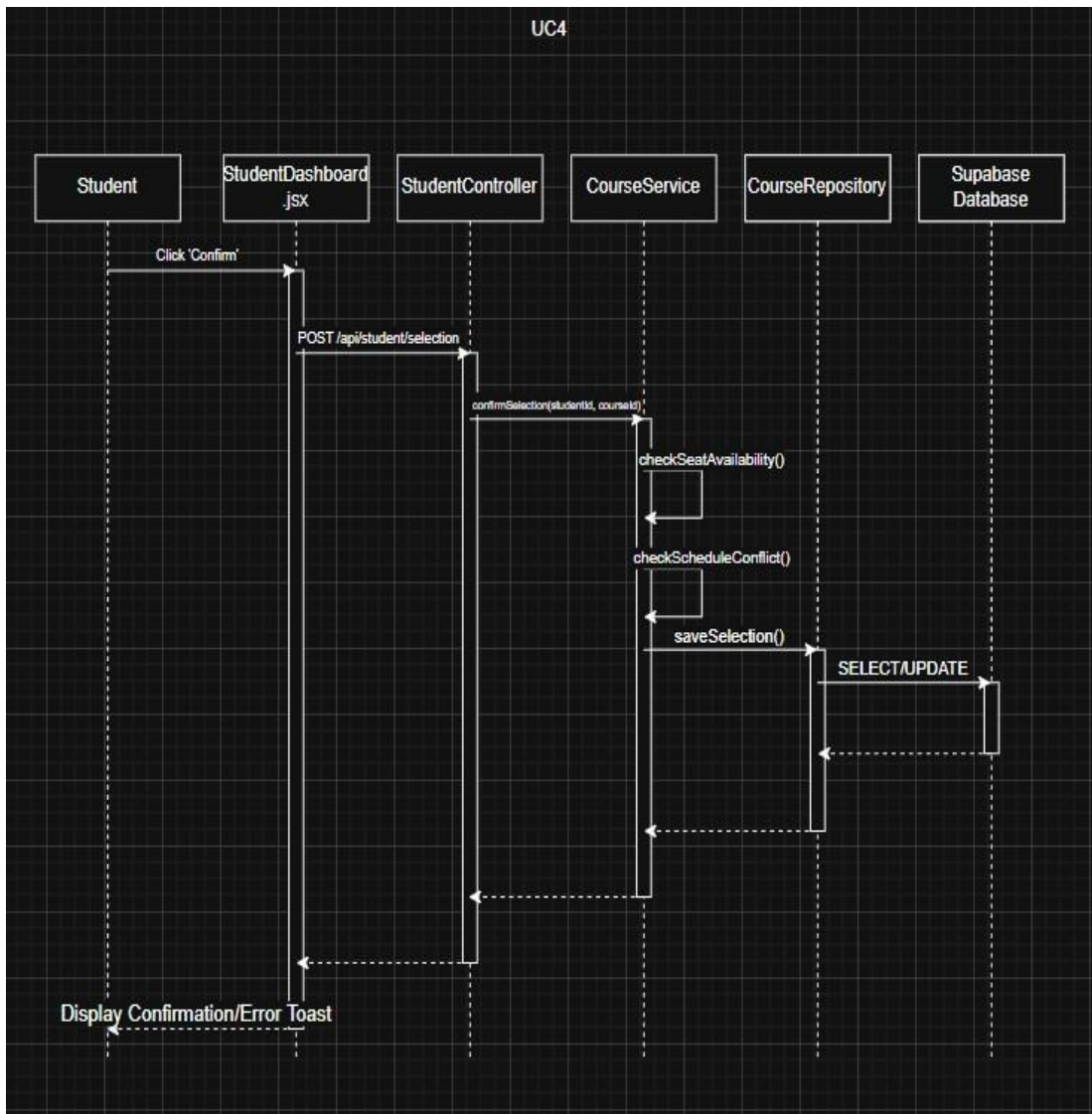


4.2 Use Case Realization

This section describes how the key use cases identified in Section 2 are implemented through the collaboration of specific software classes and modules. The following realizations trace the logic flow from the user interface down to the persistence layer.

4.2.1 Realization of UC4: Preview & Confirm Elective Selection

This realization covers the core student registration workflow, ensuring that all academic and capacity constraints are enforced by the backend before a seat is secured.



Realization Description:

- **Trigger:** The process is initiated when the StudentDashboard.jsx sends a POST request to the StudentController.
- **Validation Logic:** The CourseService acts as the primary orchestrator. It performs an internal **Seat Limiter Check** and a **Conflict Checker** (schedule check) using its own business logic methods. These are mandatory checks that ensure the student is eligible for the specific elective.
- **Error Handling:** As shown in the alt fragment of the diagram, if either check fails, the service aborts the transaction and returns a specific error message (e.g., "Schedule Clash" or "Course Full") to the UI.

- **Outcome:** On a successful pass, the CourseRepository is invoked to **decrement the available seat count** and create a persistent record in the Supabase database.

5 Data View

This section explains how data is organized, stored, and managed in our system. Since our project (SERS — Student Elective Registration System) is highly data-driven, this part is very important. It ensures that student details, course information, and registrations are handled efficiently and without errors.

5.1 Domain Model

The domain model represents the main entities involved in the system and how they are related to each other. These entities reflect real-world objects in our application.

The key entities in our system include:

- **Student:** Stores details like student ID, name, branch, and semester.
- **Course:** Contains course ID, course name, credits, and faculty details.
- **Faculty:** Represents instructors handling different courses.
- **Registration:** Maintains which student has selected which elective.
- **Admin:** Manages courses, approvals, and system updates.

Relationships:

- A **student** can register for multiple courses.
- A **course** can have multiple students.
- A **faculty member** can handle multiple courses.
- The **admin** controls and monitors the entire system.

This structure helps us clearly understand how different parts of the system interact with each other.

5.2 Data Model (persistent data view)

The data model defines how the above entities are stored in the database. Each entity is represented as a table, and relationships are maintained using keys.

Main Tables:

- **Student Table**
 - student_id (Primary Key)
 - name
 - email
 - branch
 - semester
- **Course Table**
 - course_id (Primary Key)

- course_name
- credits
- faculty_id (Foreign Key)
- **Faculty Table**
 - faculty_id (Primary Key)
 - name
 - department
- **Registration Table**
 - registration_id (Primary Key)
 - student_id (Foreign Key)
 - course_id (Foreign Key)
 - status
- **Admin Table**
 - admin_id (Primary Key)
 - username
 - password

This structured storage ensures data consistency and avoids duplication.

5.2.1 Data Dictionary

The data dictionary provides a brief description of each important field used in the system:

Field Name	Description
student_id	Unique ID assigned to each student
course_id	Unique identifier for each course
faculty_id	Unique ID for faculty members
registration_id	Unique ID for each registration
name	Name of the user (student/faculty)
email	Official email address
branch	Department of the student
semester	Current semester
credits	Credit value of the course
status	Registration status (approved/pending/rejected)

6 Exception Handling

6.1 Exception Handling Strategy

6.1.1 Architecture

Exception handling is implemented across all backend layers:

- **Controller:** Handles input errors (invalid JSON, missing fields)
- **Service:** Throws domain-specific exceptions (conflicts, limits)
- **Repository:** Handles database-related errors
- **Global Handler:** Centralizes all exceptions and converts them into standard API responses This

ensures proper separation of concerns.

6.1.2 Standardized Error Response Model

All exceptions are converted into a uniform JSON response structure to ensure consistency across the system:

- **success (Boolean):** Indicates whether the operation succeeded
- **message (String):** Human-readable description of the error
- **errorCode (String):** Machine-readable identifier for the error type
- **timestamp (String):** Time of occurrence

This standardization enables:

- Consistent frontend handling of errors
- Clear communication to end users
- Simplified debugging and logging

Errors are intentionally phrased in **non-technical language** to improve usability.

6.1.3 Business Logic Exceptions

Key handled exceptions include:

- **Conflict Checker:** Overlapping course timings
- **Seat Limiter:** Course capacity reached
- **Category Limiter:** Exceeding elective category limits
- **Duplicate Selection:** Same course selected multiple times

6.1.4 Input Validation and Sanitization

All incoming API requests are validated before reaching business logic:

- **Structural Validation**
 - Ensures required fields are present
 - Validates data types and formats (e.g., time fields, IDs)
- **Semantic Validation**
 - Ensures referenced entities (student, course) exist in the database
- **Security Validation**
 - Sanitizes inputs to prevent injection attacks (SQL Injection, XSS)

Invalid inputs are rejected early, reducing unnecessary computation and database load.

6.1.5 Concurrency Control and Transaction Safety

Given the high likelihood of simultaneous course selection requests, SERS enforces strict consistency mechanisms:

- **Transactional Operations**
 - Seat validation and allocation occur within a single atomic transaction
 - Ensures all-or-nothing execution
- **Race Condition Prevention**
 - Seat availability is rechecked at the database level before confirmation
- **Rollback Mechanism**
 - Any failure during processing results in a full rollback of changes This

guarantees:

- No seat over-allocation
- No partial or inconsistent state update

6.1.6 System Exceptions

Includes DB failures and runtime errors.

- Detailed logs stored internally
- Users receive generic messages

6.1.7 User-Facing Error Communication

All errors are communicated to users through:

- Non-intrusive UI components (toast notifications, alerts)
- Clear, actionable messages

Example:

- Instead of exposing internal exceptions, users receive messages such as:
 - *“This course is already full.”*
 - *“Selected course conflicts with your timetable.”*

This improves usability and reduces confusion during critical workflows.

6.1.8 Exception Logging and Follow-up Actions

Drop something like this:

Exception Logging

All exceptions are logged centrally within the Global Exception Handler. The logging mechanism captures:

- Timestamp of occurrence
- Exception type and error code
- Request context (API endpoint, user ID)
- Stack trace (for system-level exceptions only)

Logging levels are used as follows:

- **WARN** → Business logic violations (expected conditions)
- **ERROR** → System-level failures (database, runtime exceptions) This

ensures traceability while avoiding excessive log noise.

Follow-up Actions

Each exception type triggers a defined recovery or user action:

- **Conflict Checker Exception**
 - User must modify or remove overlapping selections
- **Seat Limiter Exception**
 - User selects an alternative course
- **Category Limiter Exception**
 - User removes an existing elective before adding another
- **Validation Errors**
 - User corrects input fields
- **System Exceptions**
 - User is shown a generic error message
 - Operation may be retried
 - Persistent failures require administrative intervention

6.2 Quality of Service

Performance: Optimized queries, DTOs, pagination **Consistency:** Short polling + backend as source of truth **Scalability:** 3-tier design, connection pooling, stateless services **Reliability:** Graceful failure, rollback, retry support

Security: RBAC, input sanitization, controlled error messages

Deployment: Works in restricted networks (IPv4, custom ports)

6.3 Logging and Monitoring

SERS incorporates structured logging for observability and debugging:

- **Centralized Logging**
 - Captures:
 - Exceptions
 - API requests
 - Critical operations
- **Log Attributes**
 - Timestamp
 - Error type

○ User/session context		
------------------------	--	--

- **Monitoring Integration**
 - Logs can be integrated with monitoring tools for:
 - Performance tracking
 - Failure diagnostics

7 Configurable Parameters

This table describes the simple configurable parameters (name / value pairs).

Configuration Parameter Name	Definition and Usage	Dynamic?
<Parameter name>	<Definition and Usage of Parameter>	<Yes/No>

The SERS system is designed with flexibility and scalability in mind by externalizing key configuration settings. These parameters are not hardcoded into the application and can be modified without changing the source code, allowing easy deployment across different environments (development, testing, production).

7.1 Application-Level Configuration

These parameters control the core behavior of the backend application:

- **Server Port:** Defines the port on which the backend server runs. This allows deployment in environments where specific ports are required.
- **Application Environment Mode:** Specifies whether the system is running in development, testing, or production mode. This affects logging levels, debugging features, and error visibility.
- **Logging Configuration:** Controls the level of logs generated (e.g., INFO, DEBUG, ERROR) and determines where logs are stored (console or log files).

7.2 Database Configuration

These parameters define how the backend connects to the PostgreSQL database:

- **Database Connection URI:** Specifies the host, port, database name, and credentials required to connect to the database.
- **Connection Pool Settings:** Includes maximum pool size, idle timeout, and connection lifetime to ensure efficient database usage under load.
- **ORM Behavior (Hibernate Settings):** Determines how the ORM interacts with the database, such as:
 - Schema validation strategy
 - Automatic schema updates (enabled/disabled)
 - SQL query logging

7.3 Security & Access Configuration

These parameters control access and security rules:

- **CORS Allowed Origins:** Defines which frontend domains are permitted to communicate with the backend APIs.
- **Authentication Parameters:** Includes token expiration time, encryption settings, and session handling policies.
- **API Rate Limiting (Optional):** Restricts the number of requests a client can make within a given time period to prevent abuse.

7.4 Frontend Configuration

These parameters affect how the frontend interacts with the backend: • **API Base URL:** Defines the endpoint URL used by the frontend to communicate with backend services. • **Polling Interval:** Specifies how frequently the frontend refreshes seat availability and course data. • **UI Feature Toggles:** Enables or disables optional UI features (e.g., preview mode, analytics dashboard visibility).

7.5 Network & Deployment Configuration

These parameters ensure compatibility with different deployment environments: • **IPv4/IPv6 Preference:** Forces the application to use IPv4 where necessary to avoid network conflicts. • **Firewall-Compatible Ports:** Allows configuration of alternate ports for database and API communication to bypass restrictive networks. • **Cloud Deployment Settings:** Includes environment variables required for deployment platforms (e.g., Supabase connection settings).

7.6 Performance & Scalability Parameters

These parameters help maintain system performance under load: • **Query Optimization Settings:** Enables optimized JOIN fetching to prevent excessive database calls (N+1 problem). • **Caching Configuration (Optional):** Defines caching strategies for frequently accessed data such as course lists. • **Thread Pool Settings:** Controls the number of concurrent backend processes handling user requests.

8 Quality of Service

8.1 Availability

The SERS system is designed for **99.5% availability** during peak academic registration windows.

- **Resilience Design:** The Spring Boot backend is **stateless**, allowing for horizontal scaling if student traffic spikes. We utilize **HikariCP** for connection pooling to ensure the database remains responsive under heavy concurrent loads.
- **Availability Impacts:**
 - * **Batch Updates:** Admin bulk-uploads of course catalogs are processed using transactional blocks to prevent table locking, ensuring students can still "Read" the catalog during updates.
 - **Maintenance:** Housekeeping tasks (archiving old selection data) are scheduled for off-peak hours (02:00–04:00) to minimize impact.
 - **Network Resilience:** The JVM is configured for **IPv4 resolution** and alternative port bindings (e.g., 5432/443) to bypass restrictive university fir

8.2 Security and Authorization

SERS implements a **Role-Based Access Control (RBAC)** strategy to protect sensitive student data and academic integrity.

- **Authentication:** Managed via secure session tokens. The admins, faculty, and students tables serve as the distinct identity providers.
- **Authorization Qualifiers:**
 - **Student:** Access is restricted to personal profile data and the ability to modify their own elective_selections.
 - **Faculty:** Scoped access; professors can only view student rosters for courses where their FacultyID is assigned.
 - **Admin:** Full CRUD permissions on the course catalog and the **Manual Override** capability.
- **Data Security:** All API endpoints are protected against **Cross-Site Request Forgery (CSRF)**, and the backend whitelists only the official React frontend domain via **CORS** policy.

8.3 Load and Performance Implications

The system is optimized to handle high-velocity transactional bursts (the "First 5 Minutes" of registration).

- **Transaction Execution:** To meet performance requirements, the **Conflict Checker** and **Seat Limiter** are executed as in-memory service logic before hitting the database, ensuring sub-100ms response times for registration attempts.
- **Database Growth:** We project linear growth in the elective_selections and allocations tables. To maintain performance, these tables will be indexed by student_id and course_id.
- **Message Processing:** The frontend utilizes **Short-Polling** (every 30 seconds) to update seat availability. To prevent server strain, the backend uses **Optimized JOIN fetching** to retrieve seat counts in a single query rather than multiple N+1 hits.

8.4 Monitoring and Control

SERS provides administrators with visibility into system health and registration friction.

- **Measurable Values (Metrics):**
 - **Seat Fill Rates:** Real-time percentage of elective capacity vs. enrolment.
 - **Conflict Frequency:** Monitoring how often the Conflict Checker rejects a request (helps identify poorly scheduled courses).
 - **API Latency:** Tracking the response time of the confirm Registration endpoint.
- **Controllable Processes:**
 - **Connection Pool Monitor:** A daemon that tracks active vs. idle database connections.
 - **Standardized Logging:** All business logic violations (e.g., FULL_SEATS, TIME_CLASH) are logged with unique IDs, allowing for rapid debugging of student complaints without needing to restart the application.